

INTERNSPARK TASK 3 REPORT

Name

B Kundhana

Task Title

Adult Income Prediction API Deployment Using Flask and Docker

Objective

The objective of this project is to deploy a trained Machine Learning model as a REST API using Flask and containerize the application using Docker for easy deployment and execution across different environments.

Dataset Used

Adult Income Dataset

Tools and Technologies

- Python
- Flask
- Pandas
- NumPy
- Scikit-Learn
- Decision Tree Classifier
- Docker
- GitHub

Methodology

Model Building

- Loaded the Adult Income Dataset.
- Performed data preprocessing and feature selection.
- Trained a Decision Tree Classifier model.
- Saved the trained model for deployment.

API Development

A Flask API was created to serve predictions from the trained model.

Home Endpoint

Endpoint:

GET /

Purpose:

Checks whether the API is running successfully.

Response:

Adult Income Prediction API Running

Prediction Endpoint

Endpoint:

POST /predict

Purpose:

Accepts input features in JSON format and returns the predicted income class.

Implementation:

- Receives input data using request.json.
- Converts the input into a Pandas DataFrame.
- Uses the trained model to generate predictions.
- Returns the prediction as a JSON response.

Sample Request

```
{  
  "age": 39,  
  "education_num": 13,  
  "hours_per_week": 40  
}
```

Sample Response

```
{  
  "prediction": 1  
}
```

Docker Containerization

A Dockerfile was created to containerize the Flask application.

Dockerfile

FROM python:3.11

WORKDIR /app

COPY . .

RUN pip install -r requirements.txt

EXPOSE 5000

CMD ["python", "app.py"]

Requirements

flask

pandas

numpy

scikit-learn

Running the Project

Build Docker Image

docker build -t adult-income-api .

Run Docker Container

docker run -p 5000:5000 adult-income-api

Run Locally

python app.py

Results

- Successfully deployed the trained Decision Tree model using Flask.
- Created REST API endpoints for model inference.
- Containerized the application using Docker.
- Verified API functionality through local testing and sample requests.

GitHub Repository Link

<https://github.com/kundhana0712/InternSpark-Task3>

Example

```
curl -X POST http://localhost:5000/predict
-H "Content-Type: application/json"
-d '{"age":39,"education_num":13,"hours_per_week":40}'
```

Output

```
{
"prediction": 1
}
```

TASK3_INTERNSPARK.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

RAM Disk

Choose Files archive.zip

archive.zip(application/x-zip-compressed) - 460936 bytes, last modified: 25/6/2026 - 100% done
Saving archive.zip to archive.zip

[5] df = pd.read_csv(list(uploaded.keys())[0])

df.head()

	age	workclass	fnlwgt	education	education.num	marital.status	occupation	relationship	race	sex	capital.gain	capital.loss	hour
0	90	?	77053	HS-grad	9	Widowed	?	Not-in-family	White	Female	0	4356	
1	82	Private	132870	HS-grad	9	Widowed	Exec-managerial	Not-in-family	White	Female	0	4356	
2	66	?	186061	Some-college	10	Widowed	?	Unmarried	Black	Female	0	4356	
3	54	Private	140359	7th-8th	4	Divorced	Machine-op-inspct	Unmarried	White	Female	0	3900	
4	41	Private	264663	Some-college	10	Separated	Prof-specialty	Own-child	White	Female	0	3900	

Next steps: [Generate code with df](#) [New interactive sheet](#)

Variables Terminal

✓ 11:31 PM Python 3

TASK3_INTERNSPARK.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

RAM Disk

[8] from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(random_state=42)

model.fit(X_train, y_train)

DecisionTreeClassifier

DecisionTreeClassifier(random_state=42)

+ Code + Text

[9] y_pred = model.predict(X_test)

print(y_pred[:10])

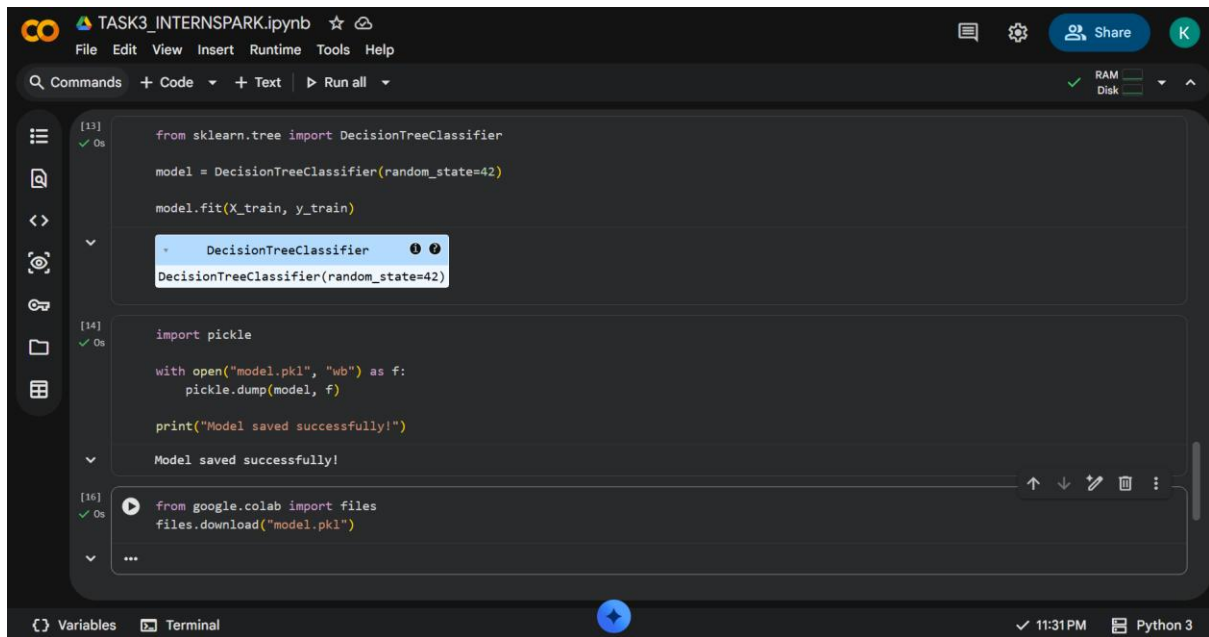
[0 0 0 0 0 0 0 0 0 0]

[10] from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

Accuracy: 0.8088438507600184



```
[13] ✓ Os
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(random_state=42)

model.fit(X_train, y_train)

DecisionTreeClassifier
DecisionTreeClassifier(random_state=42)

[14] ✓ Os
import pickle

with open("model.pkl", "wb") as f:
    pickle.dump(model, f)

print("Model saved successfully!")

Model saved successfully!

[16] ✓ Os
from google.colab import files
files.download("model.pkl")

...
```

Conclusion

This project successfully deployed an Adult Income Prediction model using Flask and Docker. The API allows users to send input data and receive income predictions through REST endpoints. Docker containerization ensures portability and simplifies deployment across multiple platforms.